CASE STUDY · PRODUCT & DESIGN

Recipe Book

Untangling a dense, overloaded costing tool – and designing Variant Recipes Management from scratch – inside a live enterprise platform, without removing a single existing feature.

ROLE	PRODUCT	SURFACE
Product / UX Designer	Hospitality-backoffice	Web application

01 – CONTEXT

A costing tool that people struggled to use

The product is a back-office platform for multi-location hospitality operators – inventory, purchasing, and food-cost management. Recipe Book is the module where operators define a "recipe" for every sellable product: the components (ingredients, packaging) that make it up, their quantities and waste, and the resulting cost and margin. Because the same product can be sold through different channels – dine-in, delivery, drive-thru, web orders – its cost and margin can differ per sales mode.

THE CONSTRAINT THAT SHAPED EVERY DECISION

This was not a greenfield redesign. Recipe Book is a live feature in a shipping enterprise product, so I worked under two hard rules: I could not remove any existing functionality, and I had to fit in an entirely new area – Variant Recipes – into the same feature. The whole project became an exercise in re-organizing a cramped, overloaded tool without taking anything away.

- Locate any product across the catalog fast.
- Manage multiple versions of a recipe.
- Edit components, quantities & waste.
- See total cost, margin and margin %.
- Compare cost & margin per sales mode.
- Model changes live, without saving.

02 – WHY REDESIGN

The signal, the reasons, and the model i looked up to

The redesign wasn't cosmetic. The clearest signal came from tech support: we had a steady stream of tickets where users complained that the screen was hard to read and navigate – they couldn't tell where they were, where a product lived or how the stacked tables related to one another.

Underneath the tickets sat a handful of concrete, defensible problems:

- browsing was hidden behind a modal, so new users had no obvious way in;
- the vertical stacking forced constant scrolling and hid the relationships a costing task depends on;
- the most valuable views – the sales mode breakdown and What if? – were buried or reachable only by accident;
- and with a whole new tab still to add, the screen was only going to get more crowded.

The model: macOS Finder's column view

To attack the root problem – **scannability** – i borrowed the layout pattern from the macOS Finder "column view" (also knowns as Miller columns). In that pattern, each selection opens the next level of the hierarchy in a new column to the right, so you move from broad to specific while every earlier choice stays visible on the screen. You never lose your place, and you can see the whole path at a glance.

Mapped onto Recipe Book, the hierarchy **category** – **product** -**recipe** – **detail** reads left to right across parallel columns. Instead of drilling into a page and losing the trail, the user keeps full context the whole time. That single principle is what makes the redesigned layout far more legible than the old stacked one – and i reused the exact same model for Variant Recipes, so both areas feel consistent and are leaned once.

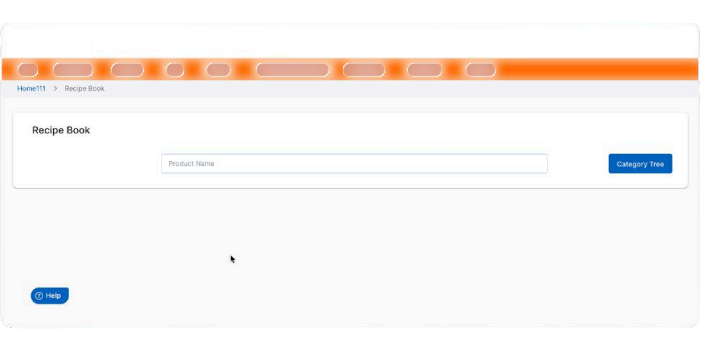
03 – THE PROBLEMS

Six things the old Recipe Book got in the way of

The feature worked – but it consistently asked users to fight the interface to do routine costing work. In order:

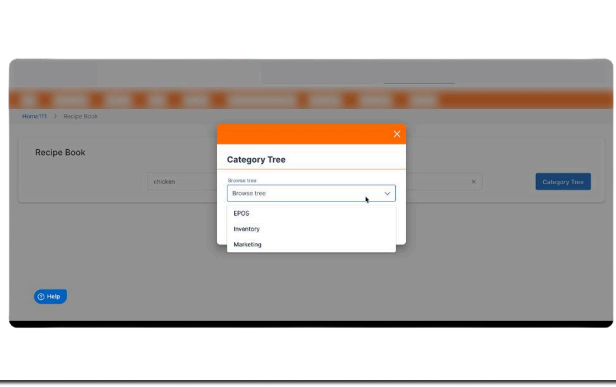
1 The landing page was a single search field

Opening Recipe Book presented one "Product Name" search input and a "Category Tree" button – nothing else. No list, no recent items, no sense of what lives inside or how to explore it. A new user has nothing to orient around; an experienced user has to already know the exact product name to make any progress. The entry point did no work to help you get started.



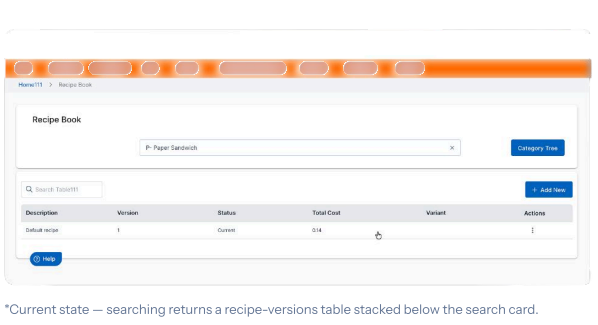
2 The main way to browse was hidden behind a button

The only way to browse by category was to open the "Category Tree" — a modal that then asks you to choose a tree (EPOS, Inventory, Marketing) from a dropdown before it shows anything at all. The primary navigation of the feature was buried two clicks deep inside a popup and split across "trees" most users don't naturally think in. Browsing — arguably the main task — was treated as a hidden, secondary action.

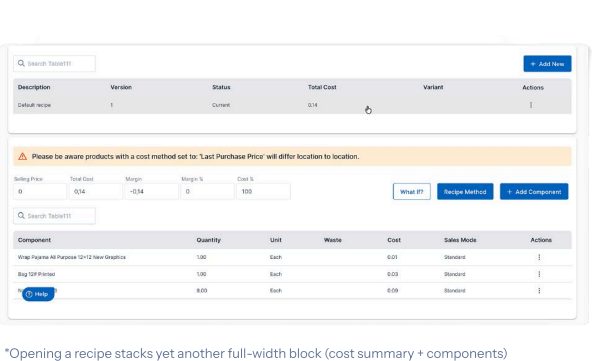


3 Everything was stacked vertically – table on table

Once you found a product, the layout stacked panels on top of one another: the recipe versions table, then the cost/margin summary, then the components table – each a fill width block. On a screen where you constantly cross-reference the recipe list, the totals, and the components, this vertical stack forces scrolling and hides the very relationships a costing task depends on. Nothing sits side by side, so nothing can be compared at a glance.



"Current state — searching returns a recipe-versions table stacked below the search card.

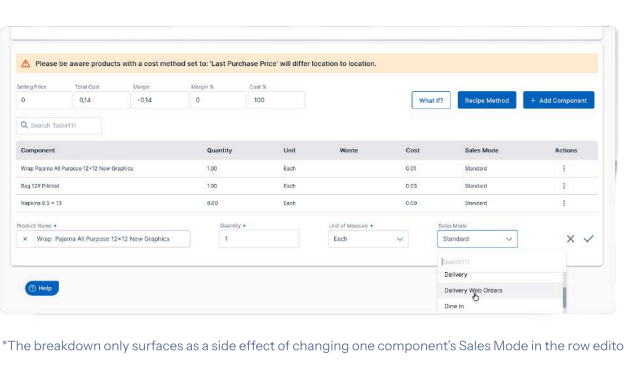


"Opening a recipe stacks yet another full-width block (cost summary + components) beneath the first, pushing content off-screen.

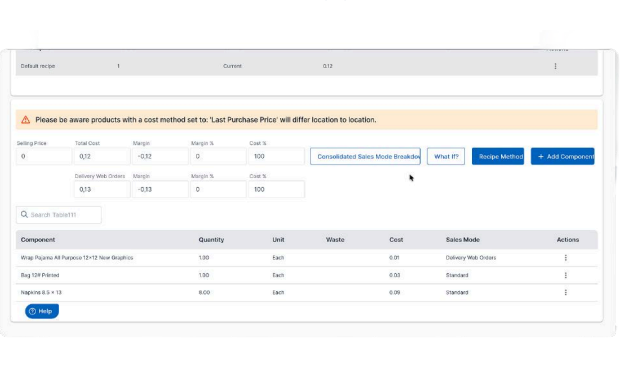
4 Consolidated Sales Mode was almost impossible to find

A product's cost can vary by sales mode, and the Consolidated Sales Mode Breakdown is the view that shows cost and margin per mode.

The problem: it only appeared after you changed the Sales Mode of an individual component in the table. There was no clear entry point – the whole breakdown was effectively gated behind and unrelated edit, so users had to stumble into it by accident rather than open it on purpose.



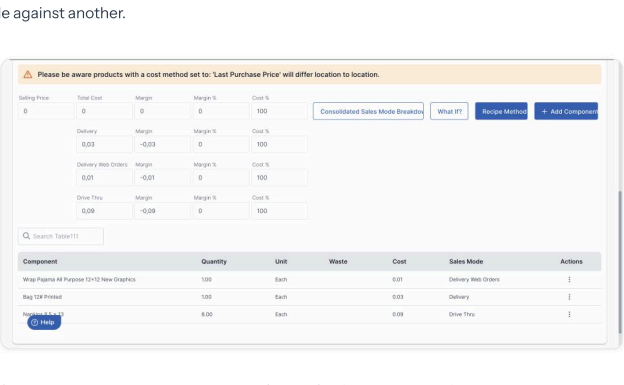
"The breakdown only surfaces as a side effect of changing one component's Sales Mode in the row editor.



"After that edit, a second "Delivery Web Orders" pricing row appears at the top.

5 ...and once triggered, the breakdown just piled up inputs

Worse, every additional sales mode you touched added another full row of Selling Price / Margin / Margin% / Cost% fields, stacked one under the next at the top of the card – Standard, then Delivery Web Orders, then Drive Thru, and so on. The more modes in play, the taller and more repetitive the header became, pushing the actual components further down and making it genuinely hard to compare one mode against another.



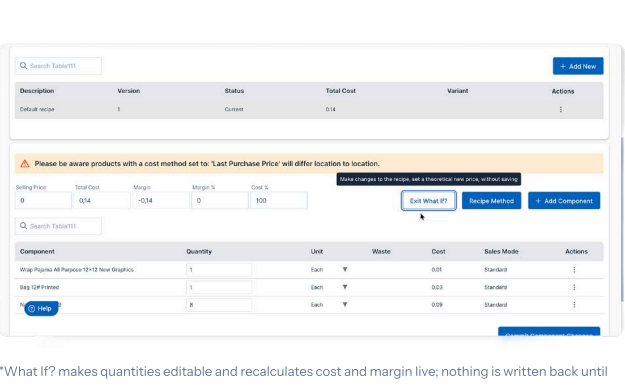
"Three sales modes in play means three stacked blocks of pricing inputs before you even reach the component table.

6 "What If?" was powerful but buried in the clutter

What is "What If?"

It's a scenario mode built into the recipe. You make temporary changes – quantities, units, components – and the cost and margin **recalculate live**, so you can see the impact of the change without saving anything to the live recipe. Component prices aren't edited by hand, they are derived from the component data and recompute automatically as you adjust the recipe.

The capability itself is genuinely useful. The problem was where it lived: trapped inside the same dense, stacked layout as everything else, with no clear signal that you'd entered a safe unsaved simulation. So the one feature meant to make experimentation feel risk free instead read as just another cramped mode – easy to miss and wit a fuzzy boundary between "trying something" and "changing the recipe".



"What If?" makes quantities editable and recalculates cost and margin live; nothing is written back until you commit.

04 – AN ENGINEERING HURDLE WE CLEARED

Search moved into the navbar – without costing us speed

Pulling search out of the page body and into the always-present navigation bar made products far more discoverable. But it changed the data contract underneath, and that raised a fair question: **would it slow things down**?

Previously the backend loaded products incrementally – one at a time as you typed, or category-by-category when you opened the Category Tree popup. With a single, always-present search in the navbar, the backend now has to know about the whole product set as soon as the page opens, so the field can search across everything immediately. On paper that sounds heavier.

In practice it isn't – and that was a deliberate call, worked out with engineering:

- The initial load pulls a lightweight index – just product names and IDs, not the full recipe or cost data – so the payload stays small.
- It loads asynchronously after first paint, so the screen is usable immediately and never blocks on the fetch.
- Once loaded it's cached for the session, so every search after that runs on the client and returns instantly – no round-trip per keystroke.

THE NET RESULT

The old design paid a network round-trip on every keystroke and every category open. The new one pays a single background load, then feels instant for the rest of the session. So even though the backend now "loads everything," perceived performance actually improved – and the discoverability win came at no speed cost. It's a good example of design and engineering solving a constraint together rather than trading one problem for another.

05 – THE PROCESS

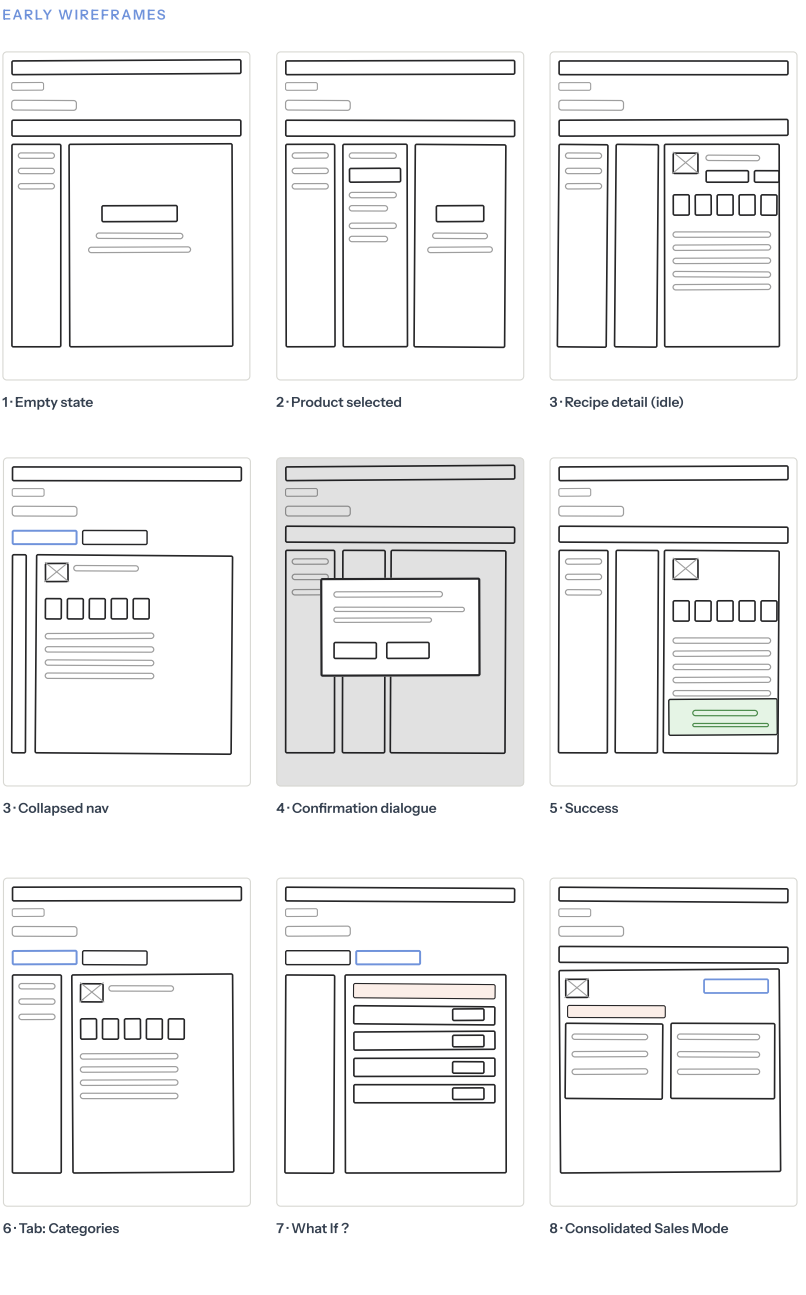
From sketch to screen

Before any high-fidelity screen existed, the column layout was tested as a wireframe. The open question wasn't visual — it was structural: would each state (nothing selected, a product selected, a recipe selected) read clearly enough that a user always knew what to click next? Low-fidelity wireframes let that get answered in an afternoon instead of a sprint.

The sketches carried the exploration across states: the empty landing state (what does a user see before they've chosen anything?), the mid-state once a product is picked but no recipe yet (do the versions read as a list or as tabs?), and the fully-loaded detail view (can cost, margin, and components genuinely coexist without the page feeling dense again?).

Alongside that, we also sketched how the same layout would need to hold up on a smaller 1024px screen – whether the sidebar should collapse first, and whether the column reveal still made sense with less horizontal room to work with. Annotations on each sketch capture the open questions at the time, not just the answers.

EARLY WIREFRAMES



1 · Empty state

2 · Product selected

3 · Recipe detail (idle)

3 · Collapsed nav

4 · Confirmation dialogue

5 · Success

6 · Tab: Categories

7 · What if ?

8 · Consolidated Sales Mode

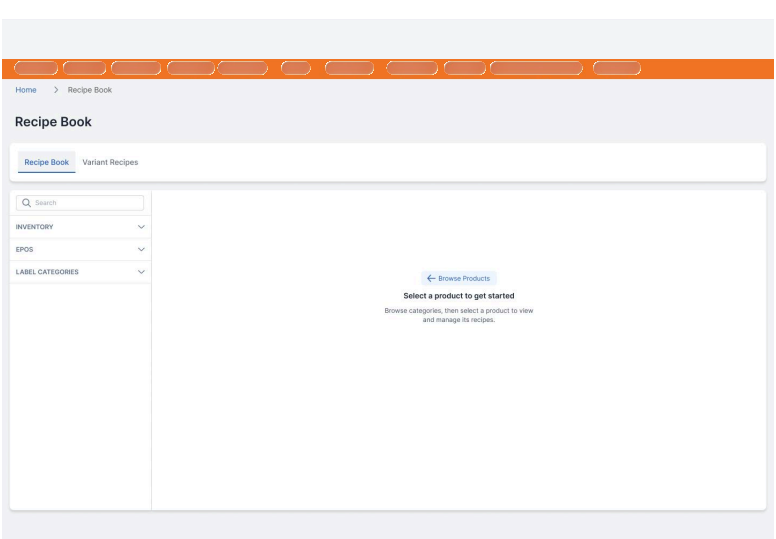
06 – THE REDESIGN

Recipe Book, reorganised – nothing removed

The redesign relocates and reframes every problem area above, while keeping all existing functionality.

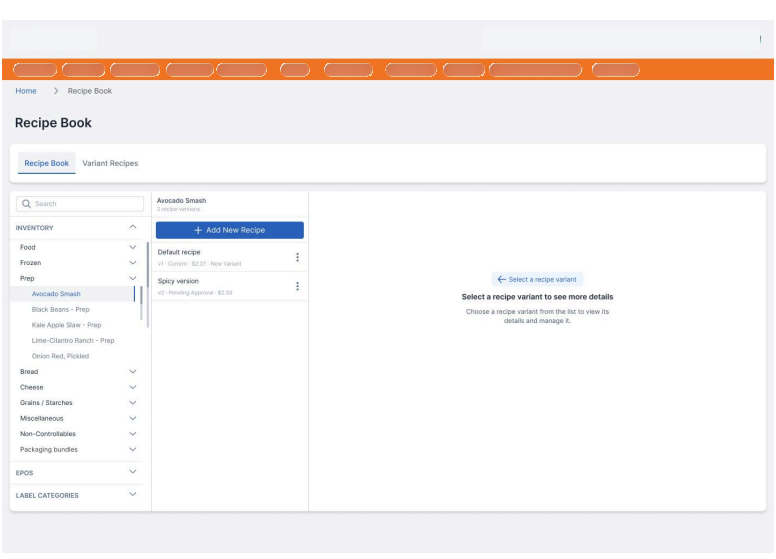
Browsing and searching move into the open, in a persistent left rail

The hidden Category Tree popup becomes a permanent left rail: search sits at the top, with category groups (Inventory, EPOS, Label Categories) always visible beneath it. Browsing and searching now live together, on the main screen, instead of behind a button – directly fixing problems 1 and 2.



A master-detail, column layout replaces the vertical stack

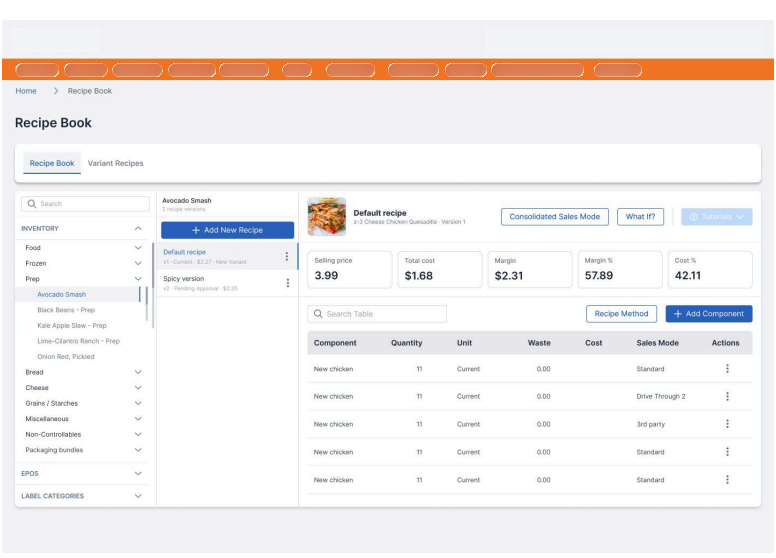
Following the Finder column model, selecting a category reveals its products, and selecting a product reveals its recipe versions in the next column – each step opening to the right while the previous choices stay in view. Related information now sits side by side instead of stacked, so nothing scrolls out of context (problem 3).



The redesigned Recipe Book, fully loaded

This is the complete screen with a product and recipe selected. The left rail keeps you oriented, the middle column lists the recipe versions, and the right panel shows everything you came for – selling price, total cost, margin and margin %, plus the full component table – all on one scannable surface.

Two tabs at the very top, **Recipe Book** and **Variant Recipes**, add the entire new capability without removing or relocating a single existing tool: new surface area, zero regressions.

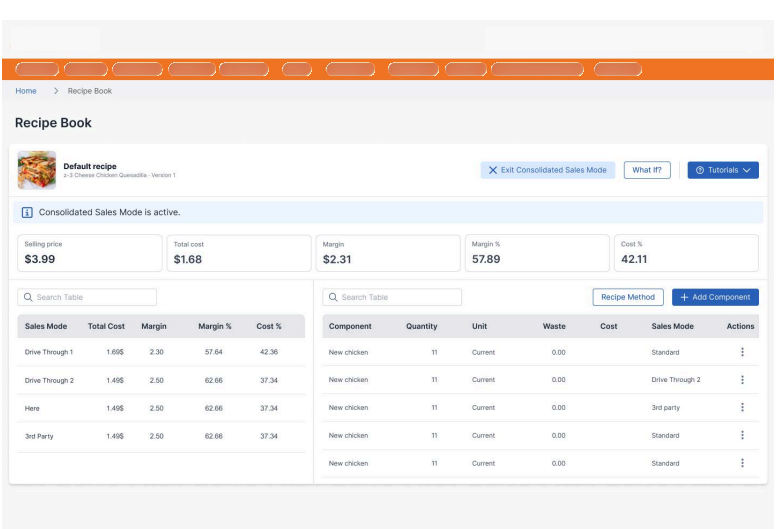


"Redesign: category → product → recipe versions read left-to-right in parallel columns; the detail panel waits on the right.

Consolidated Sales Mode becomes a deliberate action

The Sales Mode breakdown is now a clearly labeled control that users open on purpose. The button is always visible – it's simply disabled until the user has changed the sales mode of at least one component, so it's always discoverable but never fires by accident.

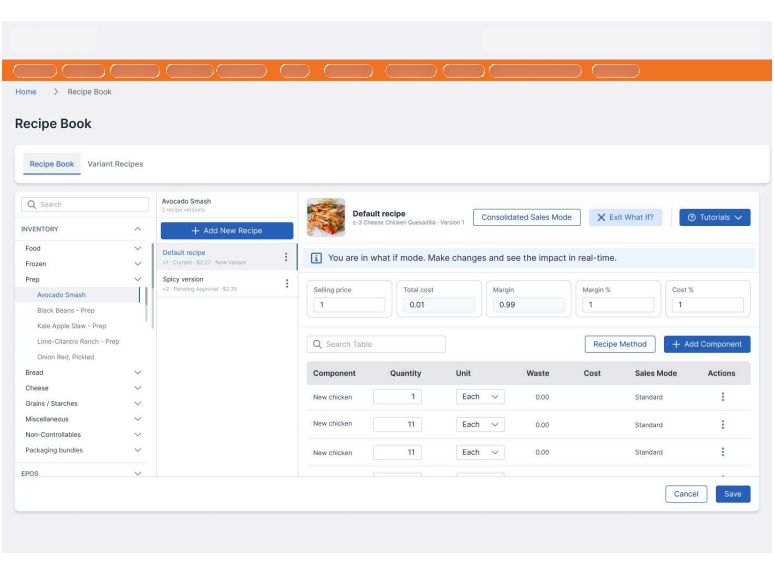
Once active, the per-mode figures are shown as a clean, side-by-side breakdown table instead of an ever-growing stack of duplicate input rows – fixing problems 4 and 5 together.



What If? becomes an unmistakable, safe simulation

What If? keeps its power but gains a clear boundary. Entering it shows an explicit banner – "You are in what if mode. Make changes and see the impact in real-time" – with an obvious way out and a Cancel / Save choice.

You adjust quantities and the cost and margin recalculate live; nothing touches the live recipe unless you deliberately save. The feature meant to make experimentation feel risk-free now actually feels that way (problem 6).



07 – THE NEW CAPABILITY

Variant Recipes Management

The new capability I designed to sit alongside Recipe Book. It solves a real problem for multi-region operators: the "same" menu item is often built differently in different places, and the system needs a way to define those variants and decide which one applies, in what order, and where.

The core idea

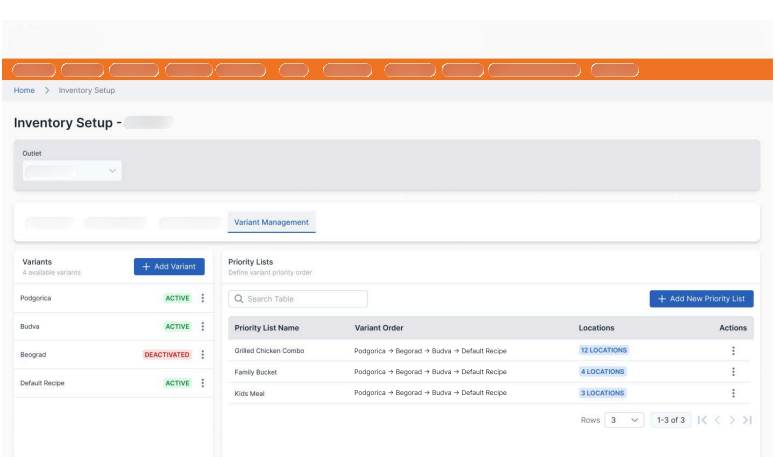
A variant is a regional or operational version of a recipe – for example a "Podgorica" versus a 'Belgrade' build of the same product. A priority list then defines the order in which variants apply for a given menu item – e.g. "Podgorica → Budva → Belgrade → Default Recipe." Together they let a business standardise a menu item centrally while still reflecting how it's genuinely made location by location.

I built this area on the same macOS-Finder column model as Recipe Book. That was a deliberate choice: users who've learned the main tool already know how to move through variants → recipes here, because the interaction is identical. One mental model covers both tabs, which shortens the learning curve, reduces mistakes, and makes a genuinely complex feature feel approachable.

Two homes, one model

Variants and their priority order are configured in Inventory Setup → Variant Management – the resulting variant recipes then surface back in Recipe Book under the Variant Recipes tab.

This is where both building blocks are created: the variants themselves and the priority lists that rank them. Configuration lives with the rest of setup, while day-to-day recipe work stays in Recipe Book.



Variants Panel

A per-location list of variants with clear Active / Deactivated status and an "Add Variant" sidebar for creating one. Variants can be added, edited, re-activated, and de-activated (with confirmation), and the design guards the obvious failure – a duplicate "same variant name", inline rather than after submission.

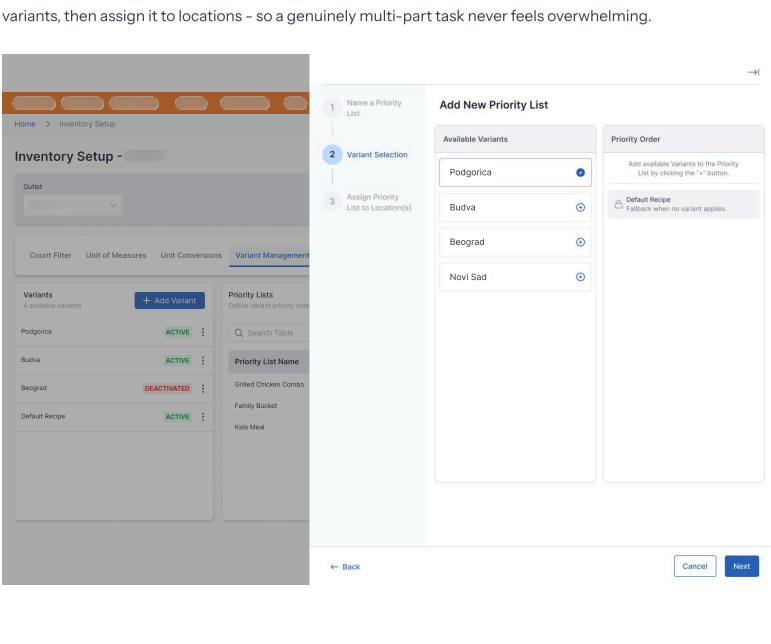
Priority Lists Panel

This is the heart of the feature. Operators can create multiple priority lists, name them, reorder variants by drag-and-drop, assign each list to locations, and delete lists.

Creation is a short, guided flow, and the design handles the messy edges explicitly, such as: multiple lists affected by a single change, same-name errors on create and edit, removing the last variant from a list, and saving a list with no locations assigned.

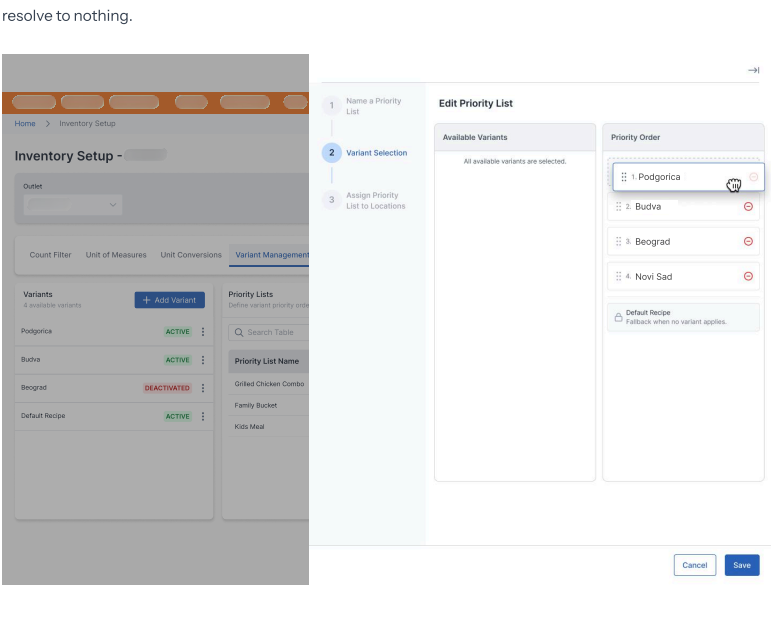
Priority List – Step 1: Naming it

The flow open as a focused side panel with a clear three-step path – name the list, choose and order variants, then assign it to locations – so a genuinely multi-part task never feels overwhelming.



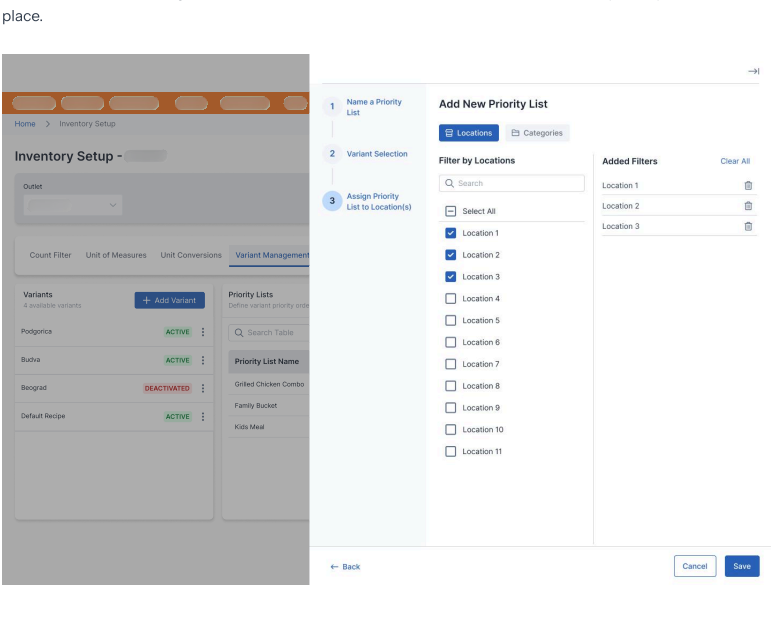
Priority List – Step 2: Choose and order variants

The user picks from the available variants on the left and builds the ranked order on the right. A locked Default Recipe always sits at the bottom as the fallback when no variant applies, so a list can never resolve to nothing.



Priority List – Step 3: Assign to locations

Finally the list is assigned to the locations it governs, filterable by location or category, with the running selection shown on the right. This is what makes the same menu item resolve differently from place to place.



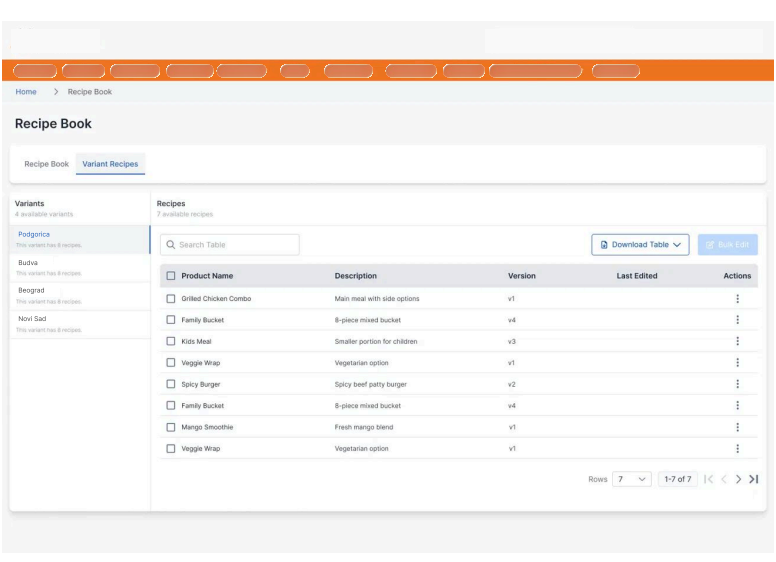
Closing the loop

Once a variant exists, the loop closes back in Recipe Book itself.

The user doesn't need to leave the module or hunt through a separate configuration screen to start attaching recipes to it – they switch to the Variant Recipes tab, pick the variant from the list on the left (Podgorica, Budva, Beograd, Novi Sad), and the right-hand panel becomes a straightforward table of every recipe assigned to that variant: product name, description, version, and when it was last edited.

Assigning more recipes is just a matter of finding them – the search field narrows the table instantly, and the checkboxes on each row let the user select several recipes at once rather than opening them one by one.

Bulk Edit is scoped deliberately: it's for adding or removing components across every selected recipe in one pass. Consider a supplier substitution, a packaging change, or an allergen that has to come out of every product that uses it – in a system without bulk actions, that's the same edit repeated once per recipe, per variant, which doesn't just cost time, it invites inconsistency.

Miss one recipe in the Podgorica variant while updating seven others and now the menu is quietly wrong in one location until someone notices, usually the hard way – a wrong cost calculation, a customer with an allergy, an audit that doesn't reconcile.